Laboratory 5
Recurrent Neural Networks

1. Recurrent Neural Networks

- **recurrent neural networks** (RNNs) are designed to better handle *sequential information* (time series, speech, text, audio, video, etc.);
- RNNs introduce state variables to store past information, together with the current inputs, to determine the current outputs;
- Preprocessing steps for text:
  1. Load text as strings into memory;
  2. Split strings into *tokens* (e.g. words and characters);
  3. Build a table of vocabulary/dictionary to map the split tokens to numerical indices starting from 0;
  4. Convert text into sequences of numerical indices so they can be manipulated by models easily.

2. Preprocessing steps for text in PyTorch

- text - 'The Time Machine' is a small **corpus** (training set) of just over 30000 words.
- task: text generation.

1. *read_time_machine()* function reads the dataset into a list of text lines, where each line is a string. For simplicity, here we ignore punctuation and capitalization.

```
def read_time_machine():
    """Load the time machine dataset into a list of text lines."""
    with open(download('http://d2l-data.s3-accelerate.amazonaws.com/timemachine.txt'), 'r') as f:
        lines = f.readlines()
    return [re.sub('[^A-Za-z]+', ' ', line).strip().lower() for line in lines]

lines = read_time_machine()
print(f'# text lines: {len(lines)}')
print(lines[0])
print(lines[10])
```

```
# text lines: 3221
the time machine by h g wells
twinkled and his usually pale face was flushed and animated the
```

2. Preprocessing steps for text in PyTorch

2. *tokenize()* function takes a list (lines) as the input and splits each text line into a list of tokens.

```
def tokenize(lines, token='word'):
    """Split text lines into word or character tokens."""
    if token == 'word':
        return [line.split() for line in lines]
    elif token == 'char':
        return [list(line) for line in lines]
    else:
        print('ERROR: unknown token type: ' + token)

tokens = tokenize(lines)
for i in range(11):
    print(tokens[i])
```

```
['the', 'time', 'machine', 'by', 'h', 'g', 'wells']
[]
[]
[]
[]
['i']
[]
[]
['the', 'time', 'traveller', 'for', 'so', 'it', 'will', 'be', 'convenient', 'to', 'speak', 'of', 'him']
['was', 'expounding', 'a', 'recondite', 'matter', 'to', 'us', 'his', 'grey', 'eyes', 'shone', 'and']
['twinkled', 'and', 'his', 'usually', 'pale', 'face', 'was', 'flushed', 'and', 'animated', 'the']
```

2. Preprocessing steps for text in PyTorch

3. the string type of the token is inconvenient to be used by models, which take numerical inputs.

```
class Vocab:
    """Vocabulary for text."""
    def __init__(self, tokens=None, min_freq=0, reserved_tokens=None):

    ...

vocab = Vocab(tokens)
print(list(vocab.token_to_idx.items())[:10])
```

```
[('<unk>', 0), ('the', 1), ('i', 2), ('and', 3), ('of', 4), ('a', 5), ('to', 6), ('was', 7), ('in', 8), ('that', 9)]
```

- we first count the unique tokens in the corpus and then assign a numerical index to each unique token according to its frequency;
- rarely appeared tokens are often removed to reduce the complexity;
- any token that does not exist in the corpus or has been removed is mapped into a special unknown token "<unk>";
- reserved tokens: <pad> – padding, <bos> – beginning of a sequence, <eos> – end of a sequence.

2. Preprocessing steps for text in PyTorch

4. *load_corpus_time_machine()* returns the corpus, a list of token indices, and vocab, the vocabulary of the time machine corpus.

```
def load_corpus_time_machine(max_tokens=-1):
    """Return token indices and the vocabulary of the time machine dataset."""
    lines = read_time_machine()
    tokens = tokenize(lines, 'char')
    vocab = Vocab(tokens)
    ...
    return corpus, vocab

corpus, vocab = load_corpus_time_machine()
len(corpus), len(vocab)
```

```
_token_freqs
[(' ', 29927), ('e', 17838), ('t', 13515), ('a', 11704), ('i', 10138), ('n', 9917), ('o', 9758)
token_to_idx
{'<unk>': 0, ' ': 1, 'e': 2, 't': 3, 'a': 4, 'i': 5, 'n': 6, 'o': 7, 's': 8, 'h': 9, 'r': 10, '
corpus
[3, 9, 2, 1, 3, 5, 13, 2, 1, 13, 4, 15, 9, 5, 6, 2, 1, 21, 19, 1, 9, 1, 18, 1, 17, 2, 12, 12, 8
(170580, 28)
```

3. RNN model

- RNNs are neural networks with hidden states;
- we consider a mini-batch of inputs (n sequence examples) $\mathbf{X_t} \in \mathbb{R}^{n \times d}$ at time step t and denote by $\mathbf{H_t} \in \mathbb{R}^{n \times h}$ the hidden variable of time step t;
- unlike the MLP, here we save the hidden variable $\mathbf{H_{t-1}}$ from the previous time step and introduce a new weight parameter $\mathbf{W_{hh}} \in \mathbb{R}^{h \times h}$ to describe how to use the hidden variable of the previous time step in the current time step;
- specifically, the calculation of the hidden variable of the current time step is determined by the input of the current time step, together with the hidden variable of the previous time step:

$$\mathbf{H_t} = \phi(\mathbf{X_t W_{xh}} + \mathbf{H_{t-1} W_{hh}} + \mathbf{b_h}). \tag{1}$$

3. RNN model

- hidden variable is called a hidden state;
- since the hidden state uses the same definition of the previous time step in the current time step, the computation of (1) is *recurrent*;
- hence, neural networks with hidden states based on recurrent computation are named **recurrent neural networks** (RNNs);
- layers that perform the computation of (1) in RNNs are called *recurrent layers*;
- for time step t, the output of the output layer is similar to the computation in the MLP:

$$\mathbf{O_t} = \mathbf{H_t W_{hq}} + \mathbf{b_q}.$$

3. RNN model

- parameters of the RNN include the weights $\mathbf{W_{xh}} \in \mathbb{R}^{d \times h}$, $\mathbf{W_{hh}} \in \mathbb{R}^{h \times h}$, and the bias $\mathbf{b_h} \in \mathbb{R}^{1 \times h}$ of the hidden layer, together with the weights $\mathbf{W_{hq}} \in \mathbb{R}^{h \times q}$, and the bias $\mathbf{b_q} \in \mathbb{R}^{1 \times q}$ of the output layer;
- it is worth mentioning that, even at different time steps, RNNs always use these same model parameters;
- therefore, the parameterization cost of an RNN does not grow as the number of time steps increases;

3. RNN model

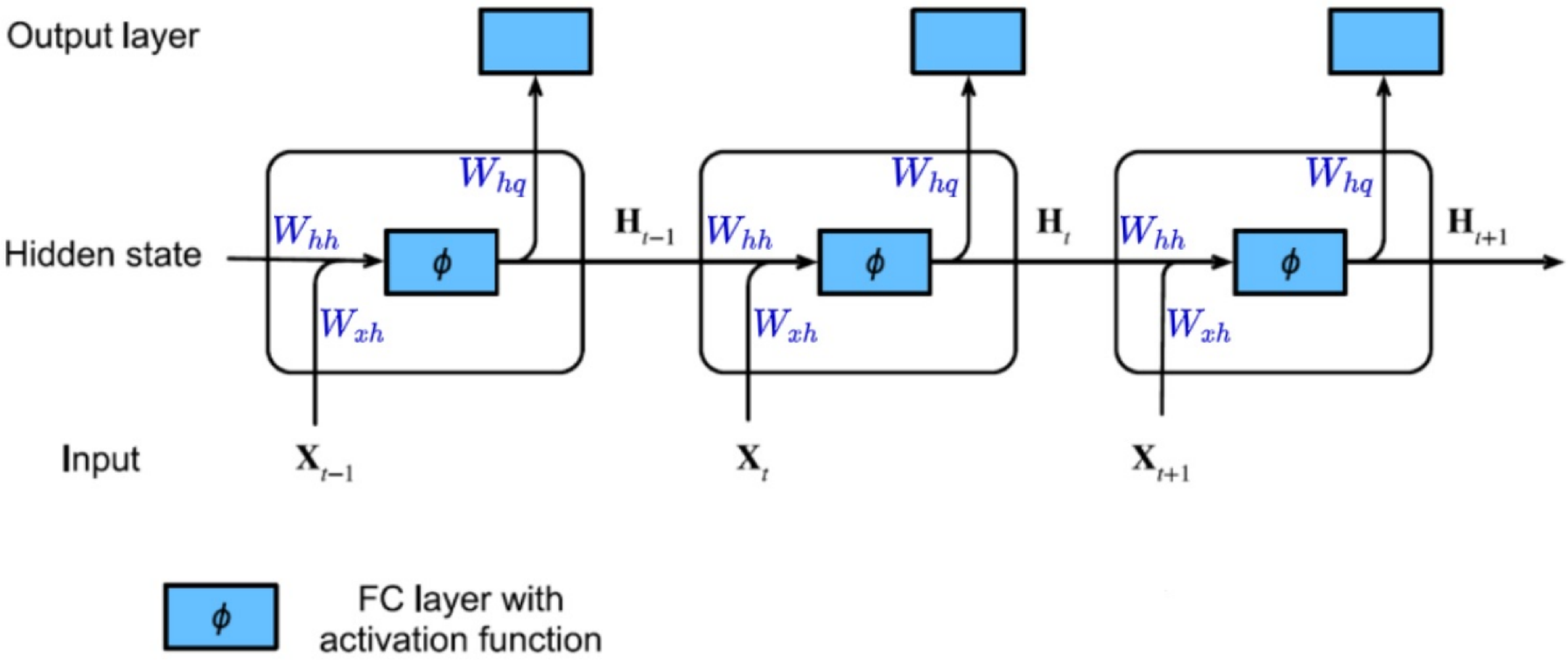


Figure 1: RNN architecture.

3. RNN model

- at any time step t, the computation of the hidden state can be treated as:
  - concatenating the input $\mathbf{X_t}$ at the current time step t and the hidden state $\mathbf{H_{t-1}}$ at the previous time step t-1;
  - feeding the concatenation result into a fully-connected layer with the activation function ϕ.
- the output of such a fully-connected layer is the hidden state $\mathbf{H_t}$ of the current time step t;
- in this case, the model parameters are the concatenation of $\mathbf{W_{xh}}$ and $\mathbf{W_{hh}}$, and a bias of $\mathbf{b_h}$, all from (1);
- $\mathbf{H_t}$ will also be fed into the fully-connected output layer to compute the output $\mathbf{O_t}$ of the current time step t;

4. RNN in PyTorch

```
num_hiddens = 256
rnn_layer = nn.RNN(len(vocab), num_hiddens)

class RNNModel(nn.Module):
    """The RNN model."""
    def __init__(self, rnn_layer, vocab_size, **kwargs):
        super(RNNModel, self).__init__(**kwargs)
        self.rnn = rnn_layer
        self.vocab_size = vocab_size
        self.num_hiddens = self.rnn.hidden_size
        self.linear = nn.Linear(self.num_hiddens, self.vocab_size)

    def forward(self, inputs, state):
        X = F.one_hot(inputs.T.long(), self.vocab_size)
        X = X.to(torch.float32)
        Y, state = self.rnn(X, state)
        output = self.linear(Y.reshape((-1, Y.shape[-1])))
        return output, state
    ...

net = RNNModel(rnn_layer, vocab_size=len(vocab))
```

- construct the recurrent neural network layer *rnn_layer* with a single hidden layer and 256 hidden units;
- convert tokens into a format that may be fed into ML algorithms using *one-hot encoding*;
- the output Y of *rnn_layer* refers to the hidden state at each time step, and this Y is used as input to the subsequent output layer.

5. Training RNNs

- RNN models often need extra help to stabilize the training process because the gradients can be quite large and the optimization algorithm may fail to converge;
- an alternative is to *clip the gradient* $\mathbf{g}$ by projecting it back to a ball of a given radius, say θ, via:

$$\mathbf{g} \leftarrow \min\left(1, \frac{\theta}{\|\mathbf{g}\|}\right)\mathbf{g}.$$

- gradient clipping provides a quick fix to the gradient exploding;
- the *perplexity* metric is used to measure the quality of a model:

$$\exp\left(-\frac{1}{n}\sum_{t=1}^{n}\log P(x_t|x_{t-1},\cdots,x_1)\right).$$

6. Training RNNs – PyTorch

```python
def load_data_time_machine(batch_size, num_steps, max_tokens=10000):
    """Return the iterator and the vocabulary of the time machine dataset."""
    data_iter = SeqDataLoader(
        batch_size, num_steps, max_tokens)
    return data_iter, data_iter.vocab

batch_size, num_steps = 32, 35
train_iter, vocab = load_data_time_machine(batch_size, num_steps)
```

➢ it returns both the data iterator and the vocabulary;
➢ num_steps → number of time steps;

```python
def grad_clipping(net, theta):
    """Clip the gradient."""
    params = [p for p in net.parameters() if p.requires_grad]
    norm = torch.sqrt(sum(torch.sum((p.grad ** 2)) for p in params))
    if norm > theta:
        for param in params:
            param.grad[:] *= theta / norm
```

➢ clip the gradients of the model;

6. Training RNNs – PyTorch

```python
def train(net, train_iter, vocab, lr, num_epochs, device):
    """Train a model."""
    loss = nn.CrossEntropyLoss()
    perplexities = []
    # Initialize
    optimizer = torch.optim.SGD(net.parameters(), lr)
    # Train and predict
    for epoch in range(num_epochs):
        ppl = train_epoch(
            net, train_iter, loss, optimizer, device)
        if (epoch + 1) % 10 == 0:
            print(predict('time traveller', 50, net, vocab, device))
            perplexities.append(ppl)
    print(f'perplexity {ppl:.1f}, device {str(device)}')
    print(predict('time traveller', 50, net, vocab, device))
    print(predict('traveller', 50, net, vocab, device))

    return perplexities

num_epochs, lr = 500, 1
perplexities = train(net, train_iter, vocab, lr, num_epochs, device)
```

```
time traveller the the the the the the the the the the the the the t
time traveller and the the the the the the the the the the the the t
time traveller the the the the the the the the the the the the the t
time traveller the this the thice some that sime time sion so di
```

7. Gated Recurrent Units (GRUs)

- RNNs may suffer from the gradient issues (vanishing gradient or exploding gradient);
- to this end, a few specific memory units have been built, namely **Long Short-Term Memory** (**LSTM**) and **Gated Recurrent Units** (**GRUs**);
- **GRUs** introduce the *reset gate* and the *update gate*;
- the *reset gate* allows us to control how much of the previous state we might still want to remember;
- the *update gate* allows us to control how much of the new state is just a copy of the old state;
- Figure 2 illustrates the inputs for both the reset and update gates in a GRU, given the input of the current time step and the hidden state of the previous time step;

7. Gated Recurrent Units (GRUs)

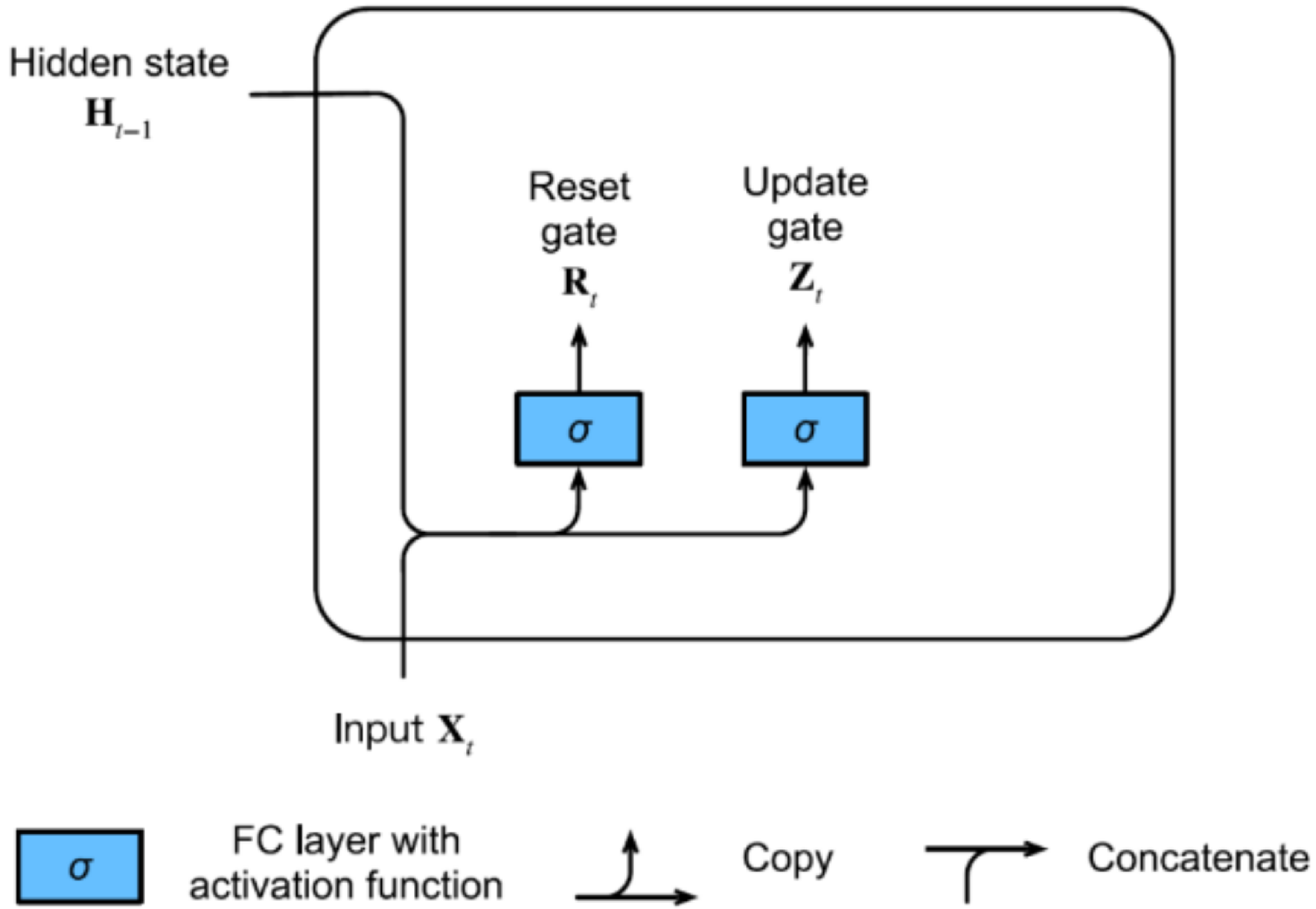


Figure 2: Computing the reset gate and the update gate in a GRU model.

7. Gated Recurrent Units (GRUs)

- the outputs of the two gates are given by two fully-connected layers with a sigmoid activation function;
- for a given time step t, the *reset gate* $\mathbf{R_t} \in \mathbb{R}^{n \times h}$ and *update gate* $\mathbf{Z_t} \in \mathbb{R}^{n \times h}$ are computed as follows:

$$\mathbf{R_t} = \sigma(\mathbf{X_t W_{xr}} + \mathbf{H_{t-1} W_{hr}} + \mathbf{b_r}),$$

$$\mathbf{Z_t} = \sigma(\mathbf{X_t W_{xz}} + \mathbf{H_{t-1} W_{hz}} + \mathbf{b_z}),$$

where $\mathbf{X_t} \in \mathbb{R}^{n \times d}$ is the input, $\mathbf{H_{t-1}}$ is the hidden state of the previous time step, $\mathbf{W_{xr}}$, $\mathbf{W_{xz}} \in \mathbb{R}^{d \times h}$ and $\mathbf{W_{hr}}$, $\mathbf{W_{hz}} \in \mathbb{R}^{h \times h}$ are weight parameters, and $\mathbf{b_r}$, $\mathbf{b_z} \in \mathbb{R}^{1 \times h}$ are biases.

7. Gated Recurrent Units (GRUs)

- integrate $\mathbf{R_t}$ in (1) to compute the *candidate hidden state* $\tilde{\mathbf{H}}_\mathbf{t} \in \mathbb{R}^{n \times h}$ at time step t:

$$\tilde{\mathbf{H}}_\mathbf{t} = tanh(\mathbf{X_t}\mathbf{W_{xh}} + (\mathbf{R_t} \odot \mathbf{H_{t-1}})\mathbf{W_{hh}} + \mathbf{b_h}),$$

where $\mathbf{W_{xh}} \in \mathbb{R}^{d \times h}$ and $\mathbf{W_{hh}} \in \mathbb{R}^{h \times h}$ are weight parameters, $\mathbf{b_h} \in \mathbb{R}^{1 \times h}$ is the bias and $\odot$ represents the Hadamard product.

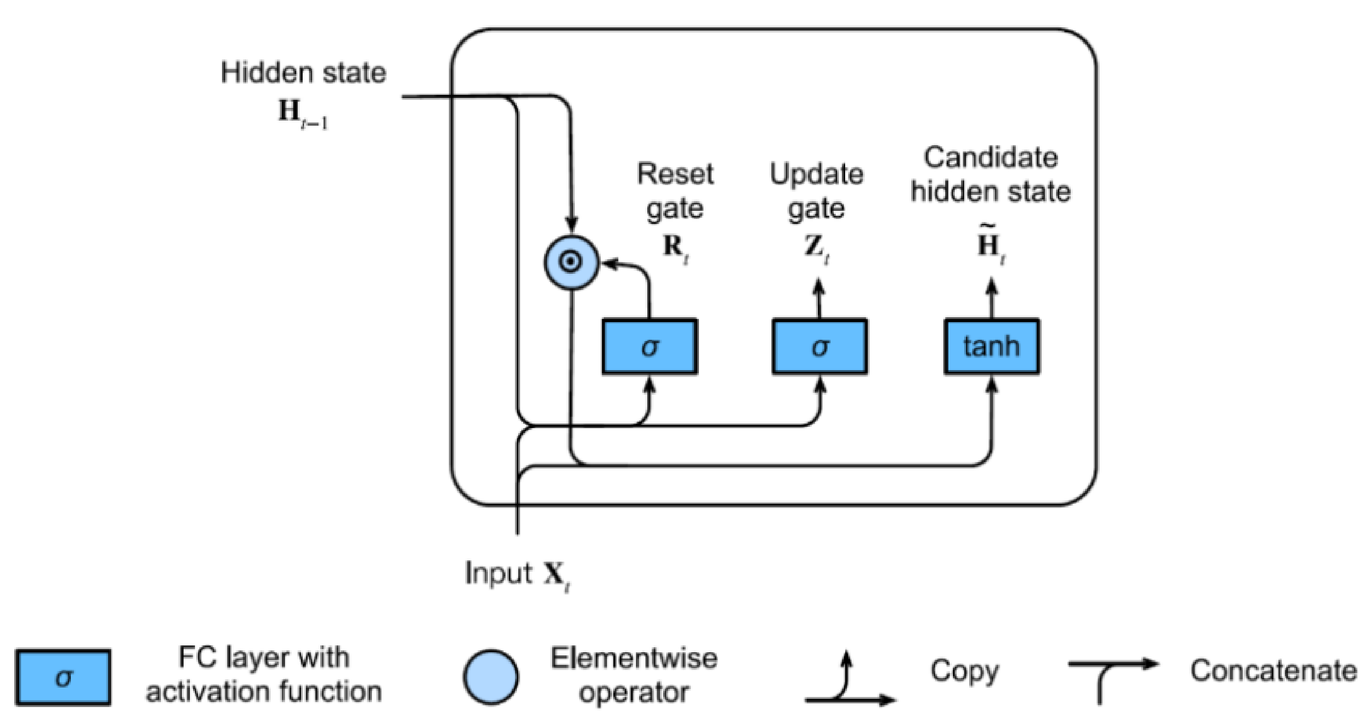


Figure 3: Computing the candidate hidden state in a GRU model.

7. Gated Recurrent Units (GRUs)

- finally, we incorporate the update gate $\mathbf{Z_t}$ to obtain the hidden state at the current time step t:

$$\mathbf{H_t} = \mathbf{Z_t} \odot \mathbf{H_{t-1}} + (1 - \mathbf{Z_t}) \odot \tilde{\mathbf{H}}_\mathbf{t}.$$

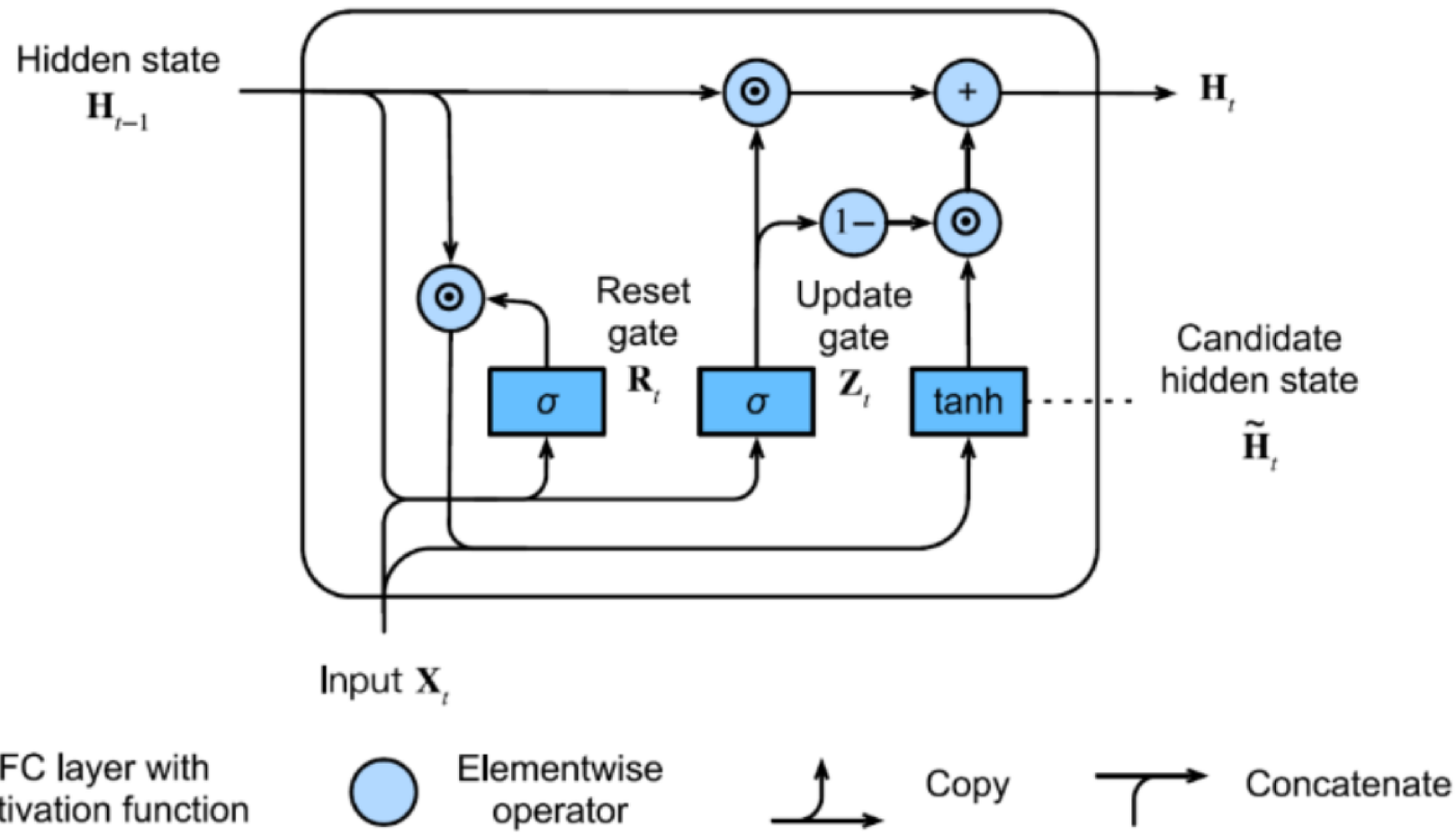


Figure 4: Computing the hidden state in a GRU model.

8. Long Short-Term Memory (LSTM)

- **LSTM** model introduces three gates, i.e., *output gate*, *input gate* and *forget gate*;
- just like in GRUs, the data feeding into the LSTM gates are the input at the current time step and the hidden state of the previous time step, Figure 5.

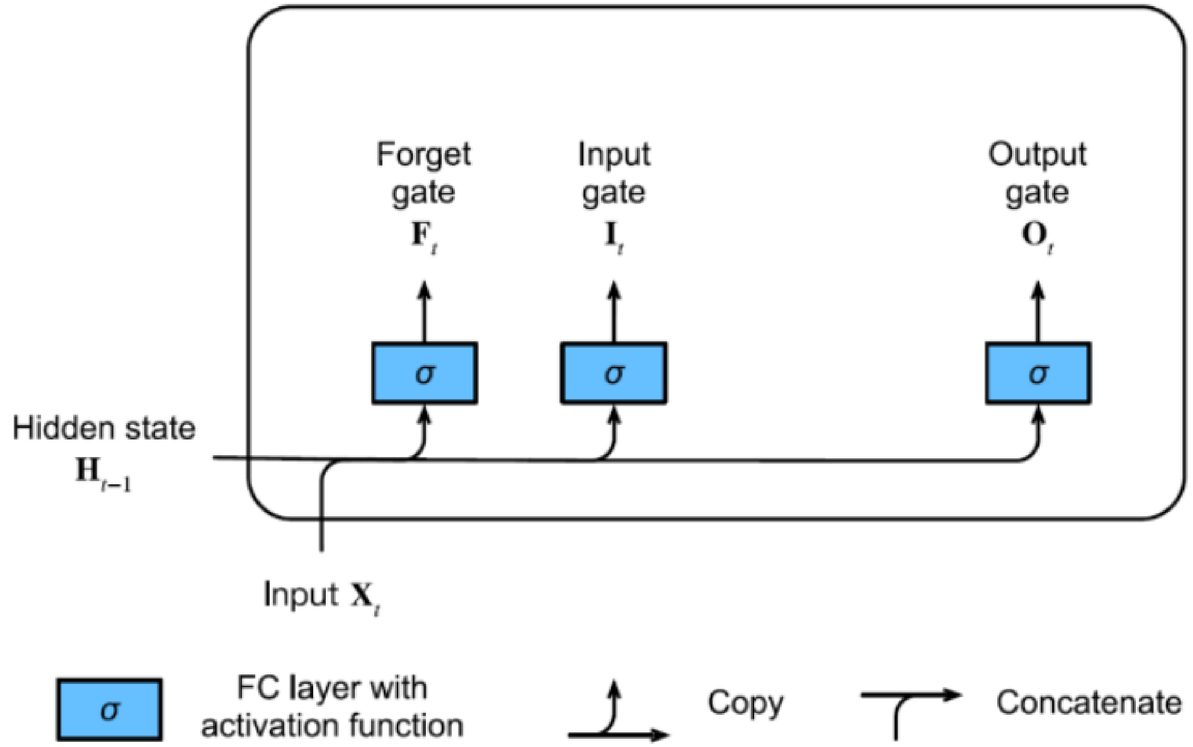


Figure 5: Computing the input gate, the forget gate, and the output gate in an LSTM model.

8. Long Short-Term Memory (LSTM)

- mathematically, the *input gate* $\mathbf{I_t}$, *forget gate* $\mathbf{F_t}$ and *output gate* $\mathbf{O_t}$ are computed as follows:

$$\begin{aligned}\mathbf{I_t} &= \sigma(\mathbf{X_t W_{xi}} + \mathbf{H_{t-1} W_{hi}} + \mathbf{b_i}),\\ \mathbf{F_t} &= \sigma(\mathbf{X_t W_{xf}} + \mathbf{H_{t-1} W_{hf}} + \mathbf{b_f}),\\ \mathbf{O_t} &= \sigma(\mathbf{X_t W_{xo}} + \mathbf{H_{t-1} W_{ho}} + \mathbf{b_o}),\end{aligned}$$

where $\mathbf{X_t} \in \mathbb{R}^{n\times d}$ is the input, $\mathbf{H_{t-1}} \in \mathbb{R}^{n\times h}$ is the hidden state at time step $t-1$, $\mathbf{W_{xi}}$, $\mathbf{W_{xf}}$, $\mathbf{W_{xo}} \in \mathbb{R}^{d\times h}$ and $\mathbf{W_{hi}}$, $\mathbf{W_{hf}}$, $\mathbf{W_{ho}} \in \mathbb{R}^{h\times h}$ are weight parameters, and $\mathbf{b_i}$, $\mathbf{b_f}$, $\mathbf{b_o} \in \mathbb{R}^{1\times h}$ are bias parameters.

- *candidate memory cell* $\tilde{\mathbf{C}}_\mathbf{t} \in \mathbb{R}^{n\times h}$ is computed as:

$$\tilde{\mathbf{C}}_\mathbf{t} = tanh(\mathbf{X_t W_{xc}} + \mathbf{H_{t-1} W_{hc}} + \mathbf{b_c}),$$

where $\mathbf{W_{xc}} \in \mathbb{R}^{d\times h}$ and $\mathbf{W_{hc}} \in \mathbb{R}^{h\times h}$ are weight parameters, and $\mathbf{b_c} \in \mathbb{R}^{1\times h}$ is the bias.

8. Long Short-Term Memory (LSTM)

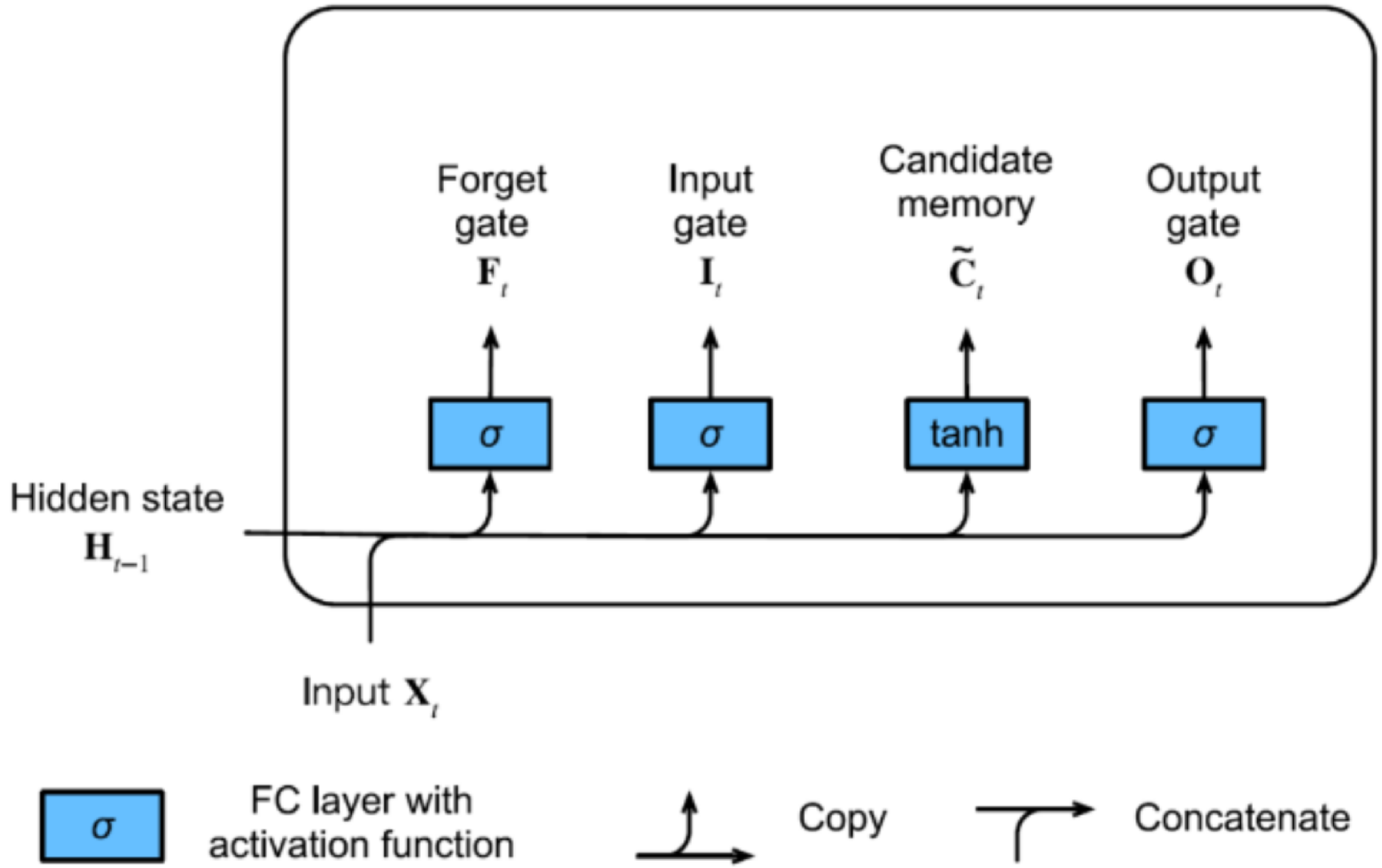


Figure 6: Computing the candidate memory cell in an LSTM model.

8. Long Short-Term Memory (LSTM)

- memory cell is computed as:

$$\mathbf{C_t} = \mathbf{F_t} \odot \mathbf{C_{t-1}} + \mathbf{I_t} \odot \tilde{\mathbf{C}}_\mathbf{t}.$$

- the hidden state in LSTM is given by:

$$\mathbf{H_t} = \mathbf{O_t} \odot tanh(\mathbf{C_t}).$$

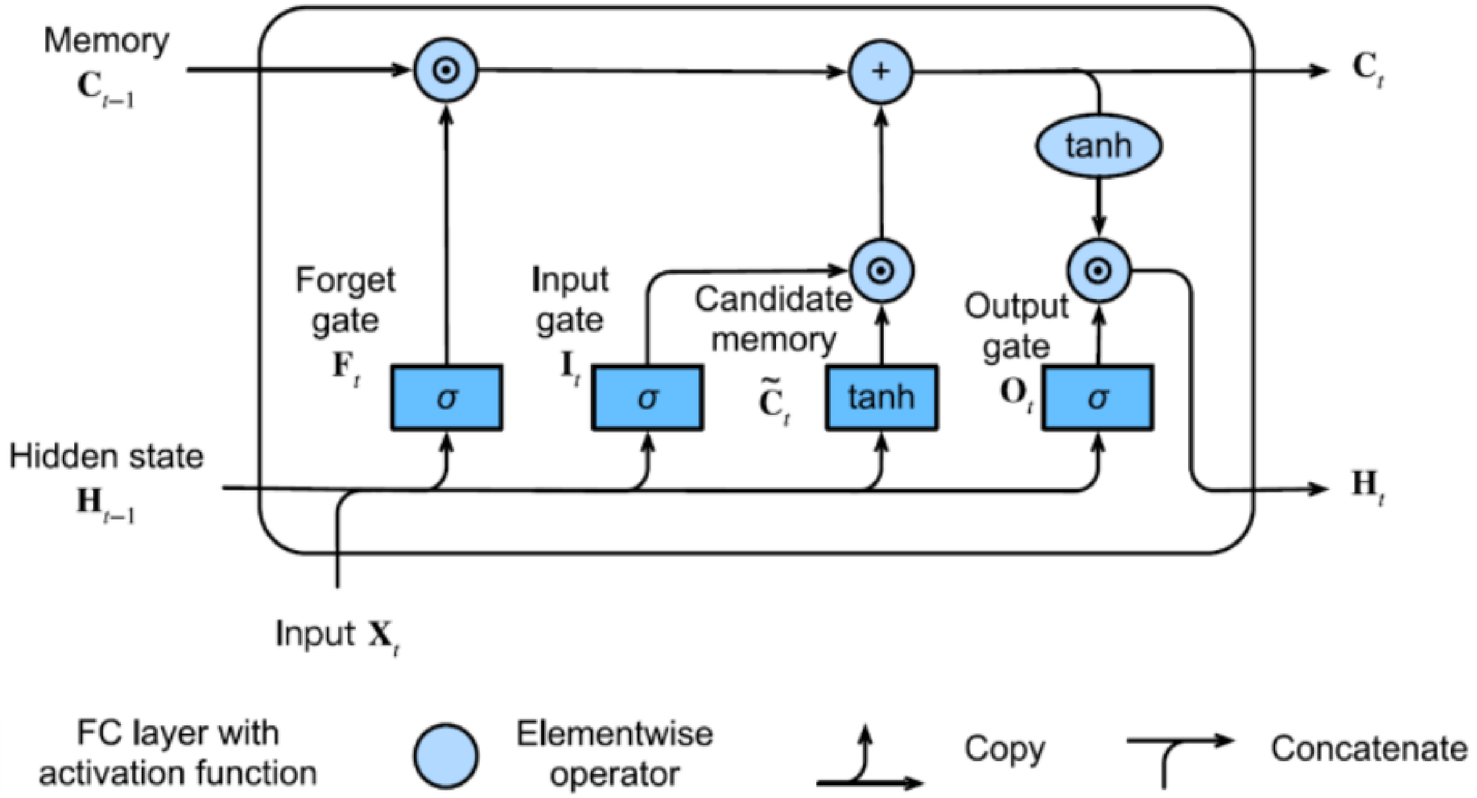


Figure 7: Computing the hidden state in an LSTM model.

9. Deep Recurrent Neural Networks

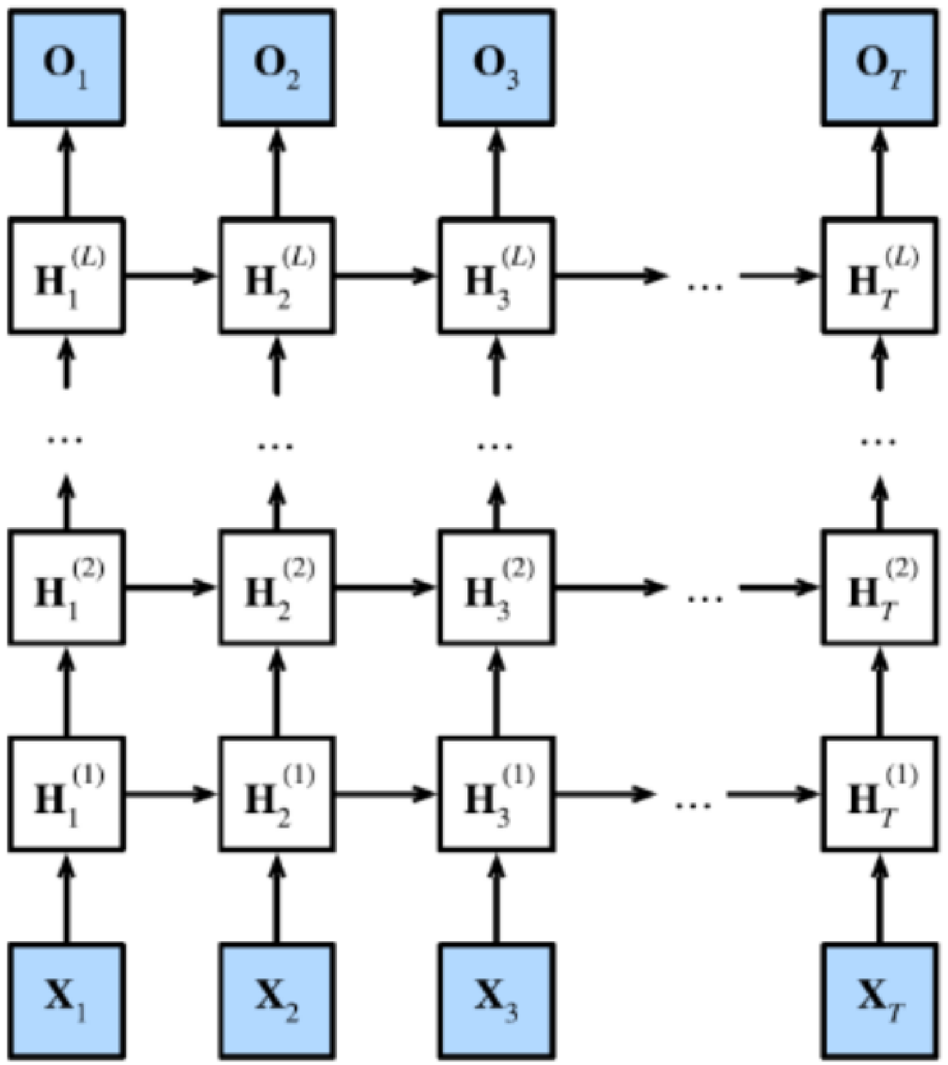


Figure 8: Deep RNN.

10. GRU, LSTM and Deep RNN in PyTorch

- *nn.GRU* layer in order to use a gated recurrent unit;

```
gru_layer = nn.GRU(num_inputs, num_hiddens)
model = RNNModel(gru_layer, len(vocab))
```

- *nn.LSTM* layer in order to use a long short-term memory;

```
lstm_layer = nn.LSTM(num_inputs, num_hiddens)
model = RNNModel(lstm_layer, len(vocab))
```

- *num_layers* specifies the number of recurrent layers;

```
num_layers = 2
lstm_layer = nn.LSTM(num_inputs, num_hiddens, num_layers)
model = RNNModel(lstm_layer, len(vocab))
```

11. Sequence to sequence learning

- NLP task: machine translation;
- machine translation datasets are composed of pairs of text sequences that are in the *source language* and the *target language*, respectively;
- for this task, the input and the output are both variable-length sequences;



Figure 9: Encoder-decoder architecture.

- *encoder*: it takes a variable-length sequence as the input and transforms it into a state with a fixed shape;
- *decoder*: it maps the encoded state of a fixed shape to a variable-length sequence.

11.1 Sequence to sequence learning in PyTorch

- English-French dataset (source language: English, target language: French);

```
def read_data_nmt():
    """Load the English-French dataset."""
    data_dir = download_extract('http://d2l-data.s3-accelerate.amazonaws.com/fra-eng.zip')
    with open(os.path.join(data_dir, 'fra.txt'), 'r') as f:
        return f.read()

raw_text = read_data_nmt()
print(raw_text[:75])
```

```
Go.     Va !
Hi.     Salut !
Run!    Cours !
Run!    Courez !
Who?    Qui ?
```

- word-level tokenization;
- create two vocabularies for both the source language and the target language separately;
- tokens that appear less than 2 times are removed;

11.1 Sequence to sequence learning in PyTorch

```python
def load_data_nmt(batch_size, num_steps, num_examples=600):
    """Return the iterator and the vocabularies of the translation dataset."""
    text = preprocess_nmt(read_data_nmt())
    source, target = tokenize_nmt(text, num_examples) # tokens = words
    src_vocab = Vocab(source, min_freq=2,
                          reserved_tokens=['<pad>', '<bos>', '<eos>']) # vocabulary for source language
    tgt_vocab = Vocab(target, min_freq=2,
                          reserved_tokens=['<pad>', '<bos>', '<eos>']) # vocabulary for target language
    src_array, src_valid_len = build_array_nmt(source, src_vocab, num_steps)
    tgt_array, tgt_valid_len = build_array_nmt(target, tgt_vocab, num_steps)
    data_arrays = (src_array, src_valid_len, tgt_array, tgt_valid_len)
    data_iter = load_array(data_arrays, batch_size)
    return data_iter, src_vocab, tgt_vocab
```

- model: two RNNs are used to design the encoder and the decoder;

```python
class Seq2SeqEncoder(Encoder):
    """The RNN encoder for sequence to sequence learning."""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0, **kwargs):
        super(Seq2SeqEncoder, self).__init__(**kwargs)
        # Embedding layer
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(embed_size, num_hiddens, num_layers,
                          dropout=dropout)

    def forward(self, X, *args):
        # The output `X` shape: (`batch_size`, `num_steps`, `embed_size`)
        X = self.embedding(X)
        X = X.permute(1, 0, 2)
        output, state = self.rnn(X)
        # `output` shape: (`num_steps`, `batch_size`, `num_hiddens`)
        # `state` shape: (`num_layers`, `batch_size`, `num_hiddens`)
        return output, state
```

11.1 Sequence to sequence learning in PyTorch

```python
class Seq2SeqDecoder(Decoder):
    """The RNN decoder for sequence to sequence learning."""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0, **kwargs):
        super(Seq2SeqDecoder, self).__init__(**kwargs)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(embed_size + num_hiddens, num_hiddens, num_layers,
                          dropout=dropout)
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, *args):
        return enc_outputs[1]

    def forward(self, X, state):
        # The output `X` shape: (`num_steps`, `batch_size`, `embed_size`)
        X = self.embedding(X).permute(1, 0, 2)
        context = state[-1].repeat(X.shape[0], 1, 1)
        X_and_context = torch.cat((X, context), 2)
        output, state = self.rnn(X_and_context, state)
        output = self.dense(output).permute(1, 0, 2)
        return output, state
```

- BLEU (Bilingual Evaluation Understudy) – originally proposed for evaluating machine translation results;

$$BLEU = \exp\left(\min\left(0, 1 - \frac{len_{label}}{len_{pred}}\right)\right) \prod_{n=1}^{k} p_n^{(1/2)^n},$$

where k is the longest n-grams for matching.

11.1 Sequence to sequence learning in PyTorch

- p_n – ratio of the number of matched n-grams in the predicted and label sequences to the number of n-grams in the predicted sequence;
- len_{label} and len_{pred} – the number of tokens in the label sequence and the predicted sequence, respectively.